F1 Race Intelligence: A Live Probabilistic Engine

CS109 Challenge Project — Ajay Shah — Stanford University — 2026

Abstract

This project is a live probabilistic system that estimates each driver's win probability in a Formula 1 race, updating after every lap. It takes in actual race data from the OpenF1 API and does a three-stage probability analysis: Gaussian MLE for corner execution scoring, Naive Bayes lap classification, and a Bayesian updater that fuses four evidence sources. It then computes, for each driver at each lap, which single move would increase their win probability the most. The system is designed as a proof-of-concept for Stanford's solar car racing team, where the same framework with telemetry variables swapped provides real-time strategic guidance during competition.

Problem Statement & Motivation

A Formula 1 race produces ≈1,100 laps of telemetry across 22 drivers, and the API samples the following data every 3 seconds: speed, throttle, brake, GPS, gap to leader, and race position. The question I want to answer is, given all observations in a lap k, what is $P(\text{winner} = i)$ for each driver i?

This felt like a natural Bayesian inference problem to me. I start with a prior over outcomes, observe sequential evidence each lap, and maintain a posterior distribution. The challenge was combining four evidence sources: position and gap are strong late-race signals, while lap time is more informative early when positions are still unsettled due to pit strategy and opening-lap incidents. After talking with Juliette about my project, I realized I could also add corner execution to my data, increasing accuracy but also the complexity of the model.

My motivation for building this is Stanford's solar car team. Solar racing involves identical strategic tradeoffs: energy conservation vs. speed, segment-by-segment benchmarking, and real-time decisions about whether to push or conserve, overtake or draft behind. F1 has so much data for practically the same probabilistic framework, and I am currently adapting this model for Solar Car and porting it to Rust so that it can take in live telemetry data during a race and return recommendations to the driver in real-time this summer. We placed 2nd last year; check back this summer, when we (hopefully) place first at the Formula Sun Grand Prix!

Data

All data is from OpenF1 (`openf1.org`), using the 2026 Australian Grand Prix (session key 11234, 56 laps, 20 race-finishing drivers).

Variables: car telemetry at 3.7 Hz (speed km/h, throttle 0–100, brake bool); lap records (number, duration, start timestamp); position snapshots (driver, position, date); interval data (gap to leader per lap).

Preprocessing: Telemetry is fetched once per driver for the full session (≈31,000 points). Each lap is sliced by timestamp:

$$\text{tel}_{d,\ell} = \{p \in \text{tel}_d \mid t_\ell \le t_p \le t_\ell + \Delta_\ell + 1\}$$

where t_ℓ is `date_start` and Δ_ℓ is lap duration. At 3.7 Hz, a typical 87 s lap yields ≈320 telemetry points. Position is matched via nearest-prior-timestamp lookup since the endpoint returns (date, position) pairs rather than (lap, position) pairs.

Methods

Stage 1: Gaussian MLE for Corner Execution

For each corner $c \in \{T1, \ldots, T16\}$ and data point $f \in \{\text{speed, throttle, brake}\}$, we fit a Gaussian via MLE on the fastest 25% of laps across all drivers. This is because I assuming that fast laps represent technically optimal execution. The MLE estimators are the closed-form sample mean and variance:

$$\hat{\mu}_{c,f} = \frac{1}{n}\sum_i x_{c,f,i}, \qquad \hat{\sigma}^2_{c,f} = \frac{1}{n}\sum_i (x_{c,f,i} - \hat{\mu}_{c,f})^2$$

A driver's corner score is the log-likelihood of their telemetry under the fitted Gaussian:

$$LL(\theta) = \sum_{i=1}^{n}\left[-\log(\sqrt{2\pi\theta_1}) - \frac{(x_i - \theta_0)^2}{2\theta_1}\right]$$

where $\theta_0 = \hat{\mu}$ (mean) and $\theta_1 = \hat{\sigma}^2$ (variance) are MLE parameters fit on the fastest 25% of laps data.

Higher score means execution closer to the optimal distribution. The lap score sums all 16 corner scores. Corner importance is quantified by how much removing each corner shifts the Naive Bayes score (Stage 2), averaged across all laps — this identifies the "highest impact corner" reported in the UI.

Stage 2: Naive Bayes Lap Classifier

We define $y \in \{\text{winner, non-winner}\}$ where "winner" labels the fastest 15% of laps across the full field. We fit Gaussian Naive Bayes via MLE ffor $P(x_{c,d} \mid y)$ for each (corner, data point) pair. Under the conditional independence assumption:

$$\log P(y \mid \mathbf{x}) = \log P(y) + \sum_{c,d} \log P(x_{c,d} \mid y)$$

normalized using logs, as we learnt in lecture, to prevent underflow. The independence assumption is clearly violated, since speed and throttle are correlated, but

the classifier still works well as a continuous "winner-likeness" score in $[0, 1]$ that I feed into Stage 3 as the corner execution evidence component.

Stage 3: Bayesian Updating

We maintain $P(\text{winner} = i)$ over all $n = 22$ drivers. After each lap k:

$$P(\text{win} = i \mid \ell_{1:k}) \;\propto\; L_k(i)^T \cdot P(\text{win} = i \mid \ell_{1:k-1})$$

$$L_k(i) = w_1 r_{\text{pos}} + w_2 r_{\text{gap}} + w_3 r_{\text{lap}} + w_4 r_{\text{corner}}$$

$$T = 0.25 + 0.30 \cdot \tfrac{k}{K}, \qquad \textstyle\sum_j w_j = 1$$

where $r_{\cdot}$ denotes rank-normalization to $[0, 1]$ across the field. Weights shift over the race: w_1 (position) ramps $0.35 \to 0.60$; w_4 (corner) decays $0.25 \to 0.05$; $w_2, w_3 = 0.20$ constant. Three key design choices require justification:

- **Rank normalization.** When I first combined the four signals, the position of the drivers, which is an integer between 1 and 22, was being drowned out by the corner log-likelihoods, which were large negative numbers. I went to AI for help and was presented with the idea rank normalization: each signal is sorted across all 22 drivers and mapped to $[0, 1]$ based on relative standing, so the best driver on any metric gets 1.0 and the worst gets 0.0. This makes all four signals on the same scale before weighting.
- **Temperature $T < 1$.** This is another technique I found through AI. Without it, 56 sequential Bayesian updates caused the posterior to collapse — one driver would reach near 100% probability within 10 laps and never recover, even if they then fell back. Raising the likelihood L to a power $T < 1$ reduceds how hard each single lap can change the posterior. T ramps upward as the race stabilizes and positions actually are decisive. Very cool technique!!
- **Laplace smoothing.** As we learnt in class, a zero likelihood permanently zeroes the posterior, and it makes recovery impossible. L is blended with ε/n ($\varepsilon = 0.03$) before tempering, ensuring no driver's likelihood is ever exactly zero. This does come with a limitation, which I address in the Limitations section.

The prior is 90% uniform +10% signal from 2025 average finishing positions ($1/\text{pos}^{1.5}$, normalized), weak enough that observed data dominates quickly.

Counterfactual Inference

This is a technique I discovered through research that was beyond the course material, but still very relevant to both CS109 content and this project. For each driver d at each lap k, I wanted to compute ΔP for four interventions by running a hypothetical Bayesian update from the current posterior with one factor modified:

$$\Delta P(d, c) = P(\text{win} = d \mid \tilde{\ell}_k) - P(\text{win} = d \mid \ell_k)$$

Interventions c: gain 1 position; cut gap by 1.5 s; match fastest lap in field; match best performer at driver's personal weakest corner.

The four ΔP values are ranked for the best advice. Counterfactual causal inference doesn't give me something that is correlated with winning the race, but gives me an actual action the driver can make that increases the chance of winning the most from where we are right now. The corner advice is the most specific: for every corner c, the system finds the best performer, computes the improvement in overall corner score if driver d matched that benchmark, runs the hypothetical update, and reports the corner with the highest ΔP as the targeted recommendation. This was the most interesting part of the entire project!

Results

Applied to the 2026 Australian Grand Prix (56 laps, 20 finishers), the system produces the following at the final lap:

- Russell (race winner, P1) reaches ≈65% win probability by end of race; Antonelli (P2) ≈28%; Leclerc (P3) ≈18%.
- Russell's probability rises sharply after lap 10 as the position weight begins to dominate, consistent with him leading cleanly after the opening stint.
- The highest-impact corner (T8) is identified consistently, matching the sector where execution variance across the field is greatest.
- Counterfactual advice for Alex Albon at lap 20 correctly identifies gaining one position as the highest-leverage intervention, and he actually does do that in the next lap, which increases his win probabilty by 5% .

Limitations

Corner segmentation divides each lap's telemetry into 16 equal time windows rather than GPS-aligned corners, which is an approximation. The Naive Bayes conditional independence assumption is violated by correlated telemetry data. Retired drivers (e.g. PIA, HUL) have posteriors frozen at retirement due to Laplace smoothing, and it does not reflect the fact that they are retired. The nearest-prior-timestamp position lookup occasionally misassigns position for laps with mid-lap position changes.

Solar Car Application

The reason I built this for F1 is that the data is free and rich. But the real goal is for Stanford's solar car: Azimuth (very cool name!). Every concept in this model maps directly — you just swap the telemetry variables:

F1 concept	Solar car equivalent
Corner execution score	Road segment efficiency
Lap time	Segment completion time
Gap to leader	Time deficit to P1
Throttle / brake	Motor current, regen, braking
Fastest 25% of laps	Most efficient 25% of segments
ΔP advice	Push vs. conserve decision

The one addition for solar is an *energy state variable* — the current battery percentage. Unlike F1 where a driver can always push harder (or at least a lot more than our little solar car), Azimuth has a hard energy budget. If you spend too much early, you may not even finish. So battery % enters the prior as a constraint and updates the probability of finishing the race at all. The counterfactual question becomes: should I push harder here, or conserve energy? The model answers that in real time.

I am currently porting this to Rust so it can ingest the live telemetry that is being sent wirelessly to an InfluxDB setup. Right now, it gets data every half-second, and my goal is to make this model as fast as possible so that during a race, a strategy member can quickly make decisions and return recommendations to the driver. We placed 2nd at the Formula Sun Grand Prix last year. The goal this summer is first.

Credits

Claude (Anthropic). I could not have built this without Claude. I had very little front-end experience going in, and Claude helped me build a majority of the GUI. Beyond that, Claude helped me see ways to apply statistical principles I already knew in ways I hadn't thought about — most notably counterfactual inference, which I would not have arrived at properly on my own. A lot of the depth in this model came from my conversations with it, which helped me not only gain new knowledge, but also improve my understanding of the existing CS109 concepts I wanted to implement..

Jeremy Hsieh. Jeremy has been working alongside me on building the telemetry pipeline for Stanford's solar car. As I port this model to Rust for real-time use this summer, having access to real solar car telemetry data is something we are figuring out together — and his contributions there have been invaluable.

Juliette Woodrow. Juliette suggested adding corner execution data to the model. That one idea meaningfully improved accuracy and made the visualization so much more interactive and interesting — the entire corner-level breakdown in the UI exists because of that conversation.

Conclusion

This project implements a probabilistic race intelligence system grounded in CS109 concepts: Gaussian MLE, Naive Bayes lap classification, sequential Bayesian updating, and the newly discovered but incredibly useful concept of counterfactual inference for actionable strategic advice. The system runs on real telemetry and produces win probability estimates consistent with the actual race result. This summer, we race at the Formula Sun Grand Prix — a chance to test this model in the real world for the first time. But that is just the warmup. In 2027, Stanford competes at the World Solar Challenge in Australia with a brand new car. The probabilistic reasoning built here will be running in the background, in real time, helping us win at the global level.